



CENTRO UNIVERSITÁRIO JORGE AMADO

Curso de Análise e Desenvolvimento de Sistemas

Lucas de Castro
Daniel Gomes
Eduardo
José Ricardo
Eli
Miguel Silva
Alan
Henrique

SIMULAÇÃO DE ECOSSISTEMA: OVELHA, LOBO E CAÇADOR

Professor Taijara

Salvador

2026

SUMÁRIO

1. INTRODUÇÃO
2. ESTRATÉGIA DA SOLUÇÃO
3. ESTRUTURAS DE DADOS UTILIZADAS
4. JUSTIFICATIVA DAS ESCOLHAS
5. PRINCIPAIS DIFICULDADES
6. CONSIDERAÇÕES FINAIS

1. INTRODUÇÃO

O presente relatório descreve o desenvolvimento de uma simulação de ecossistema em linguagem C, realizada como projeto final da disciplina de Estrutura de Dados. O sistema simula a interação entre três tipos de entidades: a ovelha, o lobo e o caçador, em um tabuleiro de tamanho configurável pelo usuário.

O objetivo principal não foi apenas desenvolver um programa funcional, mas também aplicar na prática os conceitos de estruturas de dados estudados ao longo do semestre, como matrizes, structs, ponteiros e alocação dinâmica de memória.

2. ESTRATÉGIA DA SOLUÇÃO

A simulação funciona em rodadas. A cada rodada, cada entidade se move aleatoriamente para uma célula vizinha disponível, o sistema resolve as interações entre entidades que ocupam a mesma célula e, por fim, entidades próximas de outras do mesmo tipo podem gerar novas entidades, simulando reprodução.

O tamanho do tabuleiro é informado pelo usuário no início da execução, com tamanho mínimo de 5x5. A quantidade de cada tipo de entidade e de obstáculos é calculada automaticamente com base no total de células, seguindo os percentuais definidos no enunciado do projeto: 12% para ovelhas, 12% para lobos, 5% para caçadores e 10% para obstáculos.

A simulação encerra quando uma das condições de vitória é atingida, por exemplo quando todas as ovelhas são eliminadas ou quando restam apenas entidades de um único tipo. Há também um limite máximo de 1000 rodadas para evitar que o programa rode indefinidamente.

3. ESTRUTURAS DE DADOS UTILIZADAS

O tabuleiro foi implementado como uma matriz dinâmica de inteiros, alocada com malloc. Cada célula armazena um valor inteiro que identifica o que está presente naquela posição, podendo ser vazio, ovelha, lobo, caçador, árvore ou pedra.

Cada entidade é representada por uma struct chamada Entidade, que guarda a posição atual na matriz, o tipo da entidade, um indicador de se ela está viva e um contador de rodadas usado para controlar a reprodução.

Todas as entidades são armazenadas em um vetor alocado dinamicamente. Quando a capacidade do vetor é atingida, ele é expandido com realloc, dobrando sua capacidade. Isso permite que novas entidades nasçam durante a simulação sem um limite fixo pré-definido.

4. JUSTIFICATIVA DAS ESCOLHAS

A matriz foi escolhida para o tabuleiro porque ela permite acesso direto a qualquer posição em tempo constante. Como o sistema precisa verificar células vizinhas a todo momento durante a movimentação e as interações, essa estrutura mostrou ser a mais eficiente para essa finalidade.

O vetor dinâmico de entidades foi escolhido por conta da variação no número de entidades ao longo da simulação. Mortes e nascimentos acontecem a cada rodada, então não seria adequado usar um vetor de tamanho fixo. Com realloc conseguimos ajustar a memória conforme necessário.

As structs foram usadas para agrupar os dados de cada entidade em um único lugar, o que tornou o código mais organizado e mais fácil de manter e de depurar.

5. PRINCIPAIS DIFICULDADES

A maior dificuldade foi gerenciar as interações quando duas entidades ocupavam a mesma célula após o movimento. Foi preciso cuidado para não eliminar uma entidade do tabuleiro antes de verificar todas as interações daquela rodada, evitando situações onde uma entidade já morta ainda causasse efeito sobre outra.

A reprodução também gerou dificuldades, especialmente para garantir que novas entidades nascessem somente em células realmente vazias e que o vetor de entidades fosse expandido corretamente sem comprometer os dados já existentes.

A alocação dinâmica da matriz causou alguns erros iniciais relacionados a ponteiros, principalmente na hora de liberar a memória ao encerrar o programa.

6. CONSIDERAÇÕES FINAIS

O projeto foi concluído com as funcionalidades principais implementadas e funcionando. A simulação cobre os requisitos obrigatórios do enunciado, incluindo tabuleiro configurável, proporções automáticas, dois tipos de obstáculos, movimentação aleatória, interações entre as três facções e reprodução.

O desenvolvimento do projeto ajudou o grupo a entender na prática como as decisões sobre estruturas de dados afetam a organização do código e o uso de memória, que é exatamente o objetivo da disciplina.